Operating Systems
Complete Placement Prep

For SDE Fresher Interviews — Concept-First Notes

Built from basics → advanced, with diagrams, analogies, tag-based prioritization, and interview-gold callouts.

How to Read This Document

Throughout this document you'll see colored tag pills next to section headings. Use them to prioritize your prep based on how much time you have:

Tag	What it means
MUST KNOW	Cannot skip. Will be asked in 90%+ of OS interviews.
HIGH FREQ	Asked in ~50% of OS interviews. Strong ROI.
NUMERICAL ALERT	Expect a calculation here. Practice with the worked example.
AMAZON FAV	Topics Amazon and Amazon-style interviewers love.
SERVICE-BASED FAV	Favored by Indian service companies (TCS, Infosys, Wipro, Cognizant, Capgemini).
GFG CLASSIC	Frequently appears as standalone GeeksforGeeks-style question.
RARE BUT IMPRESSIVE	Rarely asked, but stuns interviewers when you know it. Senior signal.

Additional callout types you'll see:

■ ***Analogy:*** *Yellow box. Maps an OS concept to an everyday situation. If you can recall the analogy, you can rebuild the concept under pressure.*

★ **Interview Gold:** Teal box. The high-leverage insight on a topic — the line you should be able to say in an interview to demonstrate real understanding. These are the 'don't-forget-this' takeaways.

Table of Contents

1. Introduction & The Big Picture

Before diving into processes and pages, get one mental model right: **the OS is the layer that sits between your program and the bare hardware**. Every interesting question in OS — scheduling, memory, deadlocks, file systems — is really a variation of one core problem: *multiple things want one resource at the same time*. The OS decides who gets what, when, and for how long.

■ ***Analogy:*** *Think of the OS as the manager of a busy restaurant. Customers (processes) want tables (CPU), menus (memory), waiters (I/O), and orders to be delivered fairly. The manager doesn't cook or eat — they coordinate. If two customers reach for the same plate, the manager prevents a fight. That's literally what an OS does, scaled down to microseconds.*

1.1 What does an OS actually do?

- **Process management** — creates, schedules, and kills processes.
- **Memory management** — gives each process its own private memory illusion.
- **File system** — turns disk blocks into something you can call *resume.pdf*.
- **I/O management** — talks to keyboards, disks, network cards via drivers.
- **Protection & security** — keeps process A from reading process B's memory.
- **System calls** — the API the OS exposes to user programs.

1.2 Kernel mode vs User mode

Modern CPUs have at least two privilege levels. **User mode** is restricted — your program can't directly touch hardware, can't access another process's memory, can't issue privileged instructions. **Kernel mode** can do anything. When your code needs to do something privileged (read a file, allocate memory, send a packet), it doesn't do it directly — it asks the kernel via a **system call**.

■ ***Analogy:*** *User mode is being a guest in a hotel — you can sleep, watch TV, order room service. Kernel mode is being the hotel manager — you have master keys to every room. A system call is picking up the phone and asking the front desk to do something for you. The transition (mode switch) is the elevator ride down to reception — it has a cost.*

★ **Interview Gold:** A system call is **not** the same as a function call. A function call stays in user mode. A system call causes a *trap* — a controlled mode switch into the kernel. Common syscalls: read(), write(), fork(), exec(), open(), mmap().

1.3 Types of Operating Systems (one-liners)

Type	What's special	Example
Batch	Jobs queued, no user interaction	Old IBM mainframes
Multiprogramming	Many jobs in memory; CPU never idle	Most modern OS basis
Time-sharing	Fast context switch → illusion of parallelism	Linux, Windows
Real-time (RTOS)	Hard deadlines; missing one is a failure	Pacemakers, avionics

Distributed	Many machines look like one system	Google internal infra
Embedded	Small footprint, fixed purpose	Router firmware, microwaves

1.4 The OS Boot Sequence (in 6 lines)

1. Power on → CPU jumps to a fixed ROM address (firmware: BIOS or UEFI).
2. Firmware does **POST** (Power-On Self Test) — checks RAM, devices.
3. Firmware loads the **bootloader** from disk (e.g., GRUB on Linux).
4. Bootloader loads the **kernel** into memory and jumps to it.
5. Kernel initializes drivers, mounts root FS, starts the first process (init / systemd, PID 1).
6. PID 1 spawns login, services, your shell — you're now in userland.

1.5 Why interviewers care about OS

OS questions reveal whether you understand **what's actually happening** when your code runs. A candidate who says *'I called malloc'* understands a library. A candidate who says *'malloc may trigger a brk/mmap syscall which extends the heap, possibly causing a page fault later when I touch the memory'* understands the machine. That's the gap interviewers probe.

★ **Interview Gold:** The 5 topics that show up in ~80% of fresher OS interviews: **(1) Process vs Thread, (2) Deadlock conditions, (3) Paging & Virtual Memory, (4) Scheduling algorithms with numericals, (5) Producer-Consumer / Reader-Writer synchronization.** Master these and you've covered most of the surface.

2. Processes — The Unit of Execution

MUST KNOW HIGH FREQ

A **process** is a program in execution. The program on disk is dead text — instructions sitting in a file. The moment you double-click it (or it gets forked), the OS creates a running instance with its own memory, registers, file handles, and identity. That live thing is a process.

■ ***Analogy:*** *A recipe in a cookbook is a program. The act of cooking — pots boiling, you stirring, a timer ticking — is a process. The same recipe can be 'in execution' twice (two pots of pasta on two stoves) = two processes from one program. Each has its own pot, ingredients, and progress.*

2.1 The Process Control Block (PCB)

The PCB is the OS's record of a process — a struct in kernel memory. When a process is switched out, everything needed to resume it later is stored here. Memorize the contents:

- **Process ID (PID)** — unique integer.
- **Process state** — new / ready / running / waiting / terminated.
- **Program counter (PC)** — next instruction to execute.
- **CPU registers** — saved values when not running.
- **Memory info** — page tables, base/limit registers, segments.
- **Open file descriptors** — pointers to file table entries.
- **Scheduling info** — priority, time slice used, etc.
- **Accounting** — CPU time used, user ID, parent PID.

★ **Interview Gold:** The PCB is what makes **context switching** possible. When the OS switches from process A to process B, it saves A's registers into A's PCB and loads B's registers from B's PCB. This save/restore is the cost of multitasking — usually a few microseconds, but multiplied by thousands of switches per second.

2.2 Process States

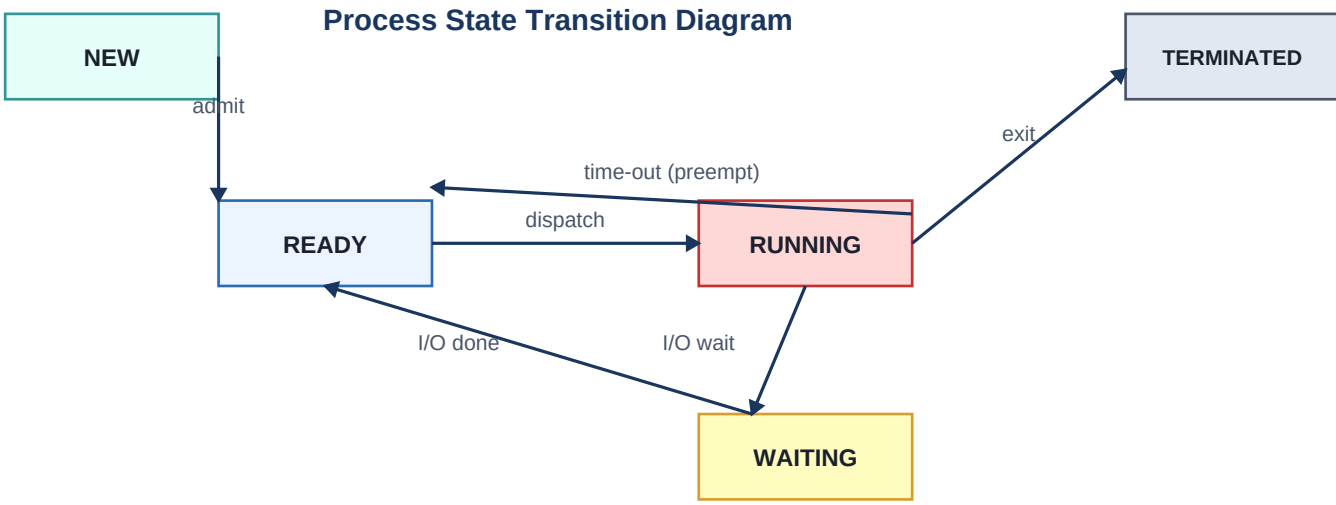
A process moves through a state machine during its life:

State	Meaning
New	Being created (PCB allocated, not yet runnable).
Ready	Loaded into memory, waiting for CPU.
Running	Currently executing on a CPU.
Waiting / Blocked	Waiting for I/O or an event (not on CPU, not eligible).
Terminated	Finished or killed; PCB will be cleaned up.

Key transitions:

- Ready → Running: scheduler dispatches it (*dispatch*).
- Running → Ready: time slice expired (*preemption*).

- Running → Waiting: process requested I/O or a lock.
- Waiting → Ready: I/O completed or lock acquired.
- Running → Terminated: exit() or killed.



■ ***Analogy:*** *Imagine a coffee shop.* ***Ready*** *= customers in line.* ***Running*** *= customer at the counter ordering.* ***Waiting*** *= customer who ordered and is waiting for their latte at the pickup area. When the latte is ready, they go back to a 'ready-to-leave' state. Time slicing is the barista saying 'next!' to keep the line moving.*

2.3 Process Memory Layout

Every process has a virtual address space carved into segments:

Segment	Grows	Contents
Text	Fixed	Compiled code (read-only, often shared).
Data	Fixed	Initialized globals/statics.
BSS	Fixed	Uninitialized globals (zero-filled at load).
Heap	Upward ↑	Dynamic memory (malloc, new).
Stack	Downward ↓	Function frames, locals, return addresses.

★ **Interview Gold:** The heap grows up; the stack grows down. They grow toward each other from opposite ends of the virtual address space. A **stack overflow** is the stack growing into a guard page. A leaking **malloc** grows the heap; eventually the OS returns NULL or kills the process (OOM).

2.4 fork(), exec(), wait(), exit()

MUST KNOW HIGH FREQ GFG CLASSIC

On UNIX, new processes are created with fork(). This is the single most asked syscall in interviews.

fork() — clone the current process

fork() creates a new process that is a near-exact copy of the parent: same code, same data, same open files, same PC. The only differences are PID and the return value of fork():

- Returns **0** in the child.
- Returns the **child's PID** in the parent.
- Returns **-1** on failure.

```
pid_t pid = fork();
if (pid == 0) {
    // child code
} else if (pid > 0) {
    // parent code
    wait(NULL);   // wait for child to finish
} else {
    perror("fork failed");
}
```

■ ***Analogy:*** *fork() is hitting 'duplicate this tab' in a browser. Both tabs are now identical, open to the same page, scrolled to the same place — but they are two independent tabs. Whatever you click in one doesn't affect the other.*

exec() — replace yourself with a new program

exec() family (execvp, execl, ...) replaces the current process image with a new program. PID stays the same, but the code, data, heap, and stack are replaced. The typical pattern is **fork-then-exec**: parent forks, child execs the new program.

wait() and exit()

exit() terminates a process. wait() lets the parent block until a child exits — and crucially, lets the parent reap the child's exit status. Without wait(), you get a **zombie**: a dead process whose PCB still lingers because nobody collected its status.

★ **Interview Gold: Zombie** = child exited, parent didn't wait(). Wastes a PCB slot. **Orphan** = parent died before child. Orphans are adopted by init (PID 1), which periodically reaps them. Both terms get asked a lot.

2.5 Inter-Process Communication (IPC)

HIGH FREQ SERVICE-BASED FAV

Processes have separate memory by default. To share data, they use IPC mechanisms. Know the trade-offs:

Mechanism	Use case	Notes
Pipe (anonymous)	Parent ↔ child, one-direction	ls | grep uses pipes.
Named pipe (FIFO)	Unrelated processes, one-direction	Has a path in the FS.
Message queue	Discrete messages, async	Kernel-managed list of msgs.
Shared memory	Fastest — direct memory access	No copy. Needs sync (mutex/sem).

Sockets	Across machines or local	Network IPC; works the same locally.
Signals	Async notification (SIGINT, etc.)	Tiny: just a number, no payload.

★ **Interview Gold:** Why is **shared memory** the fastest IPC? Because all other forms involve copying data through kernel buffers. Shared memory maps the same physical page into both processes' address spaces — after setup, no kernel involvement per access. Trade-off: you must synchronize manually.

2.6 Common interview questions on processes

• **How many processes are created by N consecutive fork() calls?** Answer: 2^N. Each fork() doubles the number of running processes. So 3 forks → 8 processes total (including the original).

• **What is copy-on-write?** After fork(), parent and child share the same physical pages marked read-only. When either writes, the OS makes a copy. Saves memory if child immediately calls exec().

• **Difference between fork() and vfork()?** vfork doesn't copy the address space — child runs in parent's space until exec/exit. Faster but dangerous.

• **What's a daemon?** A background process detached from any terminal, usually started at boot (e.g., sshd, cron).

3. Threads — Concurrency Within a Process

MUST KNOW HIGH FREQ AMAZON FAV

A **thread** is a separate flow of execution within a process. Multiple threads in one process share the address space (code, heap, globals, file descriptors) but each has its own **stack, registers, and program counter**. Threads are sometimes called *lightweight processes* — they cost less to create and switch than full processes, because there's no separate address space to set up.

■ ***Analogy:*** *A process is a house. A thread is a person living in it. Multiple people can share the kitchen, fridge, and couch (shared memory), but each has their own bed (stack) and to-do list (registers, PC). Two houses (two processes) don't share anything — to send food across, you need a delivery service (IPC).*

3.1 Process vs Thread — the table you MUST memorize

MUST KNOW HIGH FREQ AMAZON FAV SERVICE-BASED FAV

Aspect	Process	Thread
Address space	Separate per process	Shared within process
Heap, globals	Private	Shared
Stack, registers, PC	Private	Private per thread
Creation cost	High (new PCB, page tables)	Low (just a stack + thread struct)
Context switch cost	High (TLB flush, page table swap)	Low (same address space)
Communication	IPC (pipes, sockets, shm)	Just read/write shared memory
Crash isolation	One crash doesn't kill others	One thread crash → whole process dies
Use case	Strong isolation, security	Parallelism within one task

★ **Interview Gold:** Top fresher question: **Why are threads faster to context-switch than processes?** Because threads in one process share the same page tables and TLB entries. Switching threads doesn't flush the TLB. Switching processes does — and the TLB takes time to warm up again, during which every memory access is slow. That's the real cost difference.

3.2 User-level vs Kernel-level threads

User-level threads

Managed entirely by a library in user space. The kernel sees only one process and doesn't know threads exist. Switching is fast (no syscall), but if one thread does a blocking I/O call, **the entire process blocks** — because to the kernel it's one schedulable entity.

Kernel-level threads

The kernel knows about each thread and schedules them independently. One thread blocking on I/O doesn't stop others. Switching costs more (syscall to scheduler). Modern OSes (Linux, Windows) use

this.

3.3 Multithreading models

• **Many-to-One**: many user threads → one kernel thread. Fast switching but no real parallelism, and blocking kills all.

• **One-to-One**: each user thread mapped to one kernel thread. True parallelism on multicore. Used by Linux (NPTL), Windows. Cost: kernel thread creation isn't free.

• **Many-to-Many**: m user threads multiplexed over n kernel threads (n ≤ m). Best of both worlds. Hard to implement; most modern systems just use one-to-one.

3.4 Benefits of multithreading

1. **Responsiveness** — UI thread stays responsive while a background thread loads data.
2. **Resource sharing** — threads share memory by default; no IPC plumbing needed.
3. **Economy** — cheaper to create and switch than processes.
4. **Scalability** — on a 16-core machine, 16 threads can truly run in parallel.

3.5 Concurrency vs Parallelism

These two get confused constantly. Get them right:

• **Concurrency**: multiple tasks *in progress* at the same time. They may interleave on a single core via fast switching.

• **Parallelism**: multiple tasks *actually executing simultaneously* on multiple cores.

■ ***Analogy:*** *A chef juggling three pans on one stove, switching between them = concurrency. Three chefs each cooking one pan on their own stove = parallelism. You can have concurrency without parallelism (single-core OS doing time-slicing) but parallelism implies concurrency.*

3.6 Context Switch — what really happens

A context switch is when the CPU stops executing one thread/process and starts executing another. Sequence:

1. Timer interrupt fires (or thread blocks/yields).
2. CPU traps into kernel mode.
3. Kernel saves the current thread's registers and PC into its PCB/TCB.
4. Scheduler picks the next runnable thread.
5. Kernel loads the new thread's registers and PC from its PCB/TCB.
6. If it's a different process: switch page tables, flush TLB.
7. Return to user mode; new thread resumes.

★ **Interview Gold:** Context switches are **pure overhead** — no useful work happens during them. On modern hardware, a context switch is roughly 1–10 microseconds. If you switch 1000 times a second, that's 1–10 ms/sec lost to switching. Too many threads = death by context switching. This is why **thread pools** exist: reuse a small number of threads instead of constantly creating new ones.

3.7 Thread interview classics

• **Can two threads in the same process have different priorities?** Yes — kernel schedules threads, not processes, in modern one-to-one models.

• **What is a thread pool?** Pre-created set of worker threads that pick tasks from a queue. Avoids per-task thread creation overhead. Used everywhere — Java's ExecutorService, .NET's thread pool, Nginx worker model.

• **What is a green thread?** A user-space thread, often M:N scheduled. Modern term: 'fiber' / 'coroutine' (Go's goroutines, Python's asyncio). Cheaper than OS threads — millions are feasible.

• **Race condition**: two threads access shared data, and the outcome depends on the timing. Fixed via synchronization (next chapter).

4. CPU Scheduling

MUST KNOW HIGH FREQ NUMERICAL ALERT GFG CLASSIC

When multiple processes are **ready**, the scheduler picks which one gets the CPU next. This is the most numerical-heavy chapter in OS — expect to compute average waiting time, turnaround time, and Gantt charts in interviews. Learn the metrics first, then the algorithms.

4.1 Key terminology

- **Burst time (BT)** — CPU time the process needs.
- **Arrival time (AT)** — when the process enters the ready queue.
- **Completion time (CT)** — when the process finishes.
- **Turnaround time (TAT)** = CT – AT. Total time from arrival to finish.
- **Waiting time (WT)** = TAT – BT. Time spent in ready queue.
- **Response time (RT)** = time of first CPU allocation – AT. Matters for interactive systems.
- **Throughput** — processes completed per unit time.

★ **Interview Gold:** Memorize: **TAT = CT – AT**, **WT = TAT – BT**. Almost every scheduling numerical reduces to drawing a Gantt chart, reading off CT for each process, then plugging into these formulas.

4.2 Preemptive vs Non-preemptive

- **Non-preemptive**: once a process gets the CPU, it keeps it until it finishes or blocks.
- **Preemptive**: scheduler can forcibly take the CPU away (timer, higher-priority arrival).

■ ***Analogy:*** *Non-preemptive scheduling is a microwave — once started, the next person has to wait for the beep. Preemptive scheduling is a parent saying 'give your sibling a turn' — the kid using the toy is interrupted regardless of being done.*

4.3 First-Come, First-Served (FCFS)

Simplest possible scheduler — a FIFO queue. Non-preemptive.

Pros: dead simple, no starvation.

Cons: suffers from the **convoy effect** — one long process makes all short processes wait. Average waiting time is high.

■ ***Analogy:*** *Convoy effect: imagine being stuck behind a slow tractor on a single-lane road. All the cars (fast processes) pile up behind the tractor (one long process), and everyone moves at tractor speed. FCFS makes this happen in CPU land.*

Example numerical (FCFS)

NUMERICAL ALERT

Process	AT	BT	CT	TAT	WT
P1	0	5	5	5	0

P2	1	3	8	7	4
P3	2	8	16	14	6
P4	3	6	22	19	13

Average WT = (0 + 4 + 6 + 13) / 4 = **5.75**. Average TAT = (5 + 7 + 14 + 19) / 4 = **11.25**.

4.4 Shortest Job First (SJF) — non-preemptive

Pick the ready process with the smallest burst time. Provably gives the **minimum average waiting time** of any scheduling algorithm — but you need to know burst times in advance (you usually don't, in practice).

Cons: starvation — a long job may never run if short jobs keep arriving.

4.5 Shortest Remaining Time First (SRTF) — preemptive SJF

Preemptive version of SJF. When a new process arrives, if its burst time is shorter than the remaining time of the currently running process, preempt. Best average waiting time among preemptive algorithms, but same starvation problem.

Example numerical (SRTF)

NUMERICAL ALERT GFG CLASSIC

Processes: P1(AT=0,BT=8), P2(AT=1,BT=4), P3(AT=2,BT=9), P4(AT=3,BT=5).

Gantt: P1(0-1) → P2 arrives with BT=4 < remaining 7 → P2(1-5) → P4(5-10) → P1(10-17) → P3(17-26).
CTs: P2=5, P4=10, P1=17, P3=26.

Process	AT	BT	CT	TAT	WT
P1	0	8	17	17	9
P2	1	4	5	4	0
P3	2	9	26	24	15
P4	3	5	10	7	2

Avg WT = (9+0+15+2)/4 = **6.5**. Avg TAT = (17+4+24+7)/4 = **13**.

4.6 Priority Scheduling

Each process gets a priority number; highest priority runs. Can be preemptive or non-preemptive. SJF is actually a special case (priority = inverse of burst time).

Problem: starvation of low-priority processes. **Solution: aging** — gradually increase the priority of waiting processes so they eventually run.

4.7 Round Robin (RR)

The default scheduler in most time-sharing systems. Each process gets a fixed **time quantum (TQ)**. If it doesn't finish, it goes to the back of the ready queue. Preemptive by design.

★ **Interview Gold:** Time quantum tuning is everything. **Too small** → too many context switches, overhead dominates. **Too large** → behaves like FCFS, response time suffers. Typical TQ: 10–100 ms.

Example numerical (RR, TQ=2)

NUMERICAL ALERT MUST KNOW

P1(AT=0,BT=5), P2(AT=1,BT=4), P3(AT=2,BT=2), P4(AT=4,BT=1).

Gantt (TQ=2): P1(0-2) → P2(2-4) → P3(4-6) → P1(6-8) → P4(8-9) → P2(9-11) → P1(11-12).
CTs: P1=12, P2=11, P3=6, P4=9.

Process	AT	BT	CT	TAT	WT
P1	0	5	12	12	7
P2	1	4	11	10	6
P3	2	2	6	4	2
P4	4	1	9	5	4

Avg WT = (7+6+2+4)/4 = **4.75**. Avg TAT = (12+10+4+5)/4 = **7.75**.

4.8 Multilevel Queue & Multilevel Feedback Queue (MLFQ)

Multilevel queue: ready queue split into several queues (e.g., system, interactive, batch), each with its own algorithm. Processes never move between queues.

MLFQ: same idea, but processes can move between queues based on behavior. Newcomers start in the highest-priority queue with a small TQ. If they use up their TQ, they're demoted (probably CPU-bound). If they yield early (I/O-bound, interactive), they stay or get promoted.

■ ***Analogy:*** *MLFQ is like a kindergarten teacher. Quiet kids who use their turn nicely (yield early = I/O-bound) get to keep their toys at the front of the line. Loud kids who hog (use full quantum = CPU-bound) get demoted to the back. Over time, the system learns who needs short bursts of interactive time vs long batch runs. Linux's CFS is a different but spiritually related approach.*

4.9 Choosing the right algorithm — when to use what

Algorithm	Best for	Watch out for
FCFS	Batch, simple systems	Convoy effect
SJF / SRTF	Min avg WT (theoretical best)	Starvation; needs BT estimate
Priority	Real-time, mixed workloads	Starvation; use aging
Round Robin	Time-sharing, fairness	TQ tuning
MLFQ	General-purpose OS	Complex to tune

★ **Interview Gold:** Common trick question: **Which is best for minimizing average response time?** Round Robin (small TQ). Which minimizes average waiting time? SRTF / SJF. Don't conflate response time (time to first CPU allocation) with waiting time (total time in ready queue).

5. Synchronization

MUST KNOW HIGH FREQ AMAZON FAV

Synchronization is what happens when shared data meets concurrent threads. Get this chapter right and you can answer most of the 'concurrency' questions in any SDE interview — Java, Python, C++, Go, they all build on the same OS primitives.

5.1 Race condition — the source of all evil

A **race condition** occurs when two or more threads access shared data and the outcome depends on the order of execution. The classic example:

```
// counter is a shared global, initially 0
// Thread A and Thread B both run:
counter = counter + 1;

// In assembly, that's:
// LOAD  R, counter   ; read
// ADD   R, 1         ; increment
// STORE R, counter   ; write
```

If both threads read counter as 0, both add 1, both write back 1 — we did two increments but the final value is 1, not 2. **Lost update.** The non-atomicity of read-modify-write is what bites you.

■ ***Analogy:*** *Two people editing the same Google Doc offline, then both syncing. Whichever saves last overwrites the other's edits. Race conditions in code happen at nanosecond scale, but the cause is identical: unsynchronized concurrent writes to shared state.*

5.2 Critical Section Problem

A **critical section** is a piece of code that accesses shared resources. Any solution to the critical section problem must satisfy three conditions:

1. **Mutual exclusion** — only one thread in the CS at a time.
2. **Progress** — if no one is in the CS and some thread wants to enter, the decision can't be postponed forever.
3. **Bounded waiting** — a thread can't be made to wait forever; there's a bound on how many times others can enter ahead of it.

5.3 Peterson's Algorithm (two-thread software solution)

Classical software-only solution for 2 threads. Uses two shared variables: flag[2] (intent to enter) and turn (whose turn it is).

```
// For thread i (other = 1 - i):
flag[i] = true;
turn = other;
while (flag[other] && turn == other) ; // busy wait
// --- critical section ---
flag[i] = false;
```

Satisfies all three conditions but only for 2 threads. Modern CPUs may reorder writes, so in practice you need memory barriers. Peterson's is mostly historical / interview content now.

5.4 Hardware support — Test-and-Set, Compare-and-Swap

Modern hardware provides atomic instructions that read-modify-write in one indivisible step. **Test-and-Set (TAS)**: atomically set a lock to 1 and return its old value. **Compare-and-Swap (CAS)**: atomically compare a memory location to an expected value; if equal, replace.

★ **Interview Gold:** CAS is the foundation of **lock-free programming**. Libraries like std::atomic, Java's AtomicInteger, and Go's sync/atomic all use CAS under the hood. Interviewers love asking *'how would you implement a thread-safe counter without locks?'* — the answer is CAS in a loop.

5.5 Mutex (Mutual Exclusion Lock)

A mutex is a binary lock: locked or unlocked. lock() waits until free, then claims; unlock() releases. Only the thread that locked it can unlock it (this is what distinguishes a mutex from a binary semaphore).

```
pthread_mutex_lock(&m);
// critical section
pthread_mutex_unlock(&m);
```

5.6 Semaphores

A semaphore is an integer counter with two atomic operations:

- **wait(S)** / **P(S)** / **down(S)**: decrement S; if S < 0, block.
- **signal(S)** / **V(S)** / **up(S)**: increment S; if S ≤ 0, wake one waiting thread.

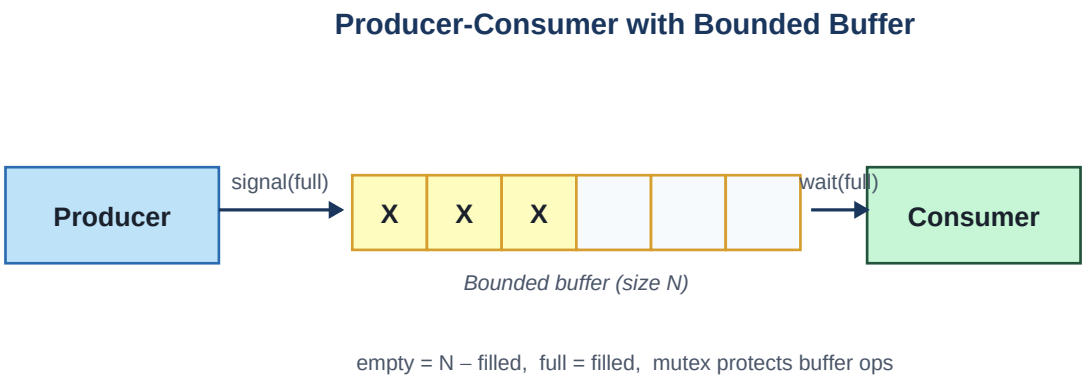
Binary semaphore — values 0 or 1. Acts like a mutex (but any thread can signal it, which is the key difference). **Counting semaphore** — values 0..N. Used to manage a pool of N resources.

■ ***Analogy:*** *A counting semaphore is the key rack at a hotel that lends bicycles. There are 10 bikes (S=10). Every guest who wants a bike does wait() — takes a key (S becomes 9, 8, ...). When all keys are gone (S=0), the next guest waits at the desk. Returning a bike is signal() — the rack gets a key back, and one waiting guest is told to come pick it up.*

5.7 Producer-Consumer (Bounded Buffer)

MUST KNOW HIGH FREQ AMAZON FAV GFG CLASSIC

Classic problem. Producers add items to a fixed-size buffer; consumers remove them. Need to ensure: (a) producer blocks when buffer is full, (b) consumer blocks when buffer is empty, (c) buffer access is atomic.




```
semaphore empty = N;       // slots free
semaphore full  = 0;       // slots filled
mutex mtx;

// Producer
wait(empty);   // wait for a free slot
lock(mtx);
add_item();
unlock(mtx);
signal(full);  // one more item ready

// Consumer
wait(full);    // wait for an item
lock(mtx);
remove_item();
unlock(mtx);
signal(empty); // freed one slot
```

★ **Interview Gold:** Why two semaphores plus a mutex? **empty** and **full** handle the counting (block when no work / no space). **mutex** protects the buffer's internal data structures (insertion index, etc). Mixing the two roles into one semaphore breaks correctness.

5.8 Reader-Writer Problem

HIGH FREQ GFG CLASSIC

Multiple readers can access shared data simultaneously — but a writer needs exclusive access. Two variants:

- **Readers-preference**: as long as readers keep coming, writers wait. Simple, but starves writers.
- **Writers-preference**: once a writer is waiting, no new readers admitted. Starves readers.
- **Fair version**: FIFO ordering; nobody starves.

5.9 Dining Philosophers

GFG CLASSIC SERVICE-BASED FAV

Five philosophers around a circular table. Each needs two forks (one on each side) to eat. Five forks total, shared between neighbors. The naive solution (each picks up left fork, then right) **deadlocks** if all five pick up their left forks simultaneously.

Common fixes:

• Allow at most 4 philosophers to attempt eating at once (limit semaphore).

• Pick up both forks atomically, or none.

• Asymmetric solution: odd philosophers pick left first, even pick right first — breaks circular wait.

■ ***Analogy:*** *Dining philosophers is the canonical illustration of the four deadlock conditions in one tiny problem. It looks silly until you realize it models distributed locks in real systems — two services each holding a resource the other needs, waiting forever.*

5.10 Monitors

A **monitor** is a higher-level synchronization construct: a class/object where only one thread can execute any of its methods at a time, automatically. Variables inside are protected; **condition variables** provide wait() and signal() for fine-grained waiting.

Java's synchronized keyword + wait()/notify() implements monitors. Same for C#'s lock. The OS provides mutexes and condition variables; monitors are the language-level abstraction.

5.11 Common interview traps

• **Mutex vs Binary Semaphore** — both ensure mutual exclusion. Key diff: mutex has *ownership* (only the locker can unlock), semaphore doesn't. Mutex usually supports priority inheritance; semaphore doesn't.

• **Spin lock vs Mutex** — spin lock busy-waits (CPU spins in a loop checking the lock). Cheap if critical section is very short and you have multiple cores. Mutex sleeps the thread — better for longer CS.

• **Why can signal() be the wrong choice over broadcast()?** If multiple threads are waiting on a condition that has changed for all of them (e.g., 'buffer not empty' after a bulk write), you want broadcast/notifyAll. Single signal can leave threads stuck.

• **Lost wakeup** — if you check a condition and then call wait() as two separate operations, a signal in between is lost. Fix: always do the check-and-wait under the same lock, using a condition variable.

6. Deadlocks

MUST KNOW HIGH FREQ AMAZON FAV GFG CLASSIC

A **deadlock** is a state where two or more processes are each waiting for a resource that another holds, and none can proceed. This chapter is interview gold — almost every fresher gets asked the four conditions and the difference between prevention and avoidance.

■ ***Analogy:*** *Two cars meet at a narrow one-lane bridge, head-on. Both refuse to back up until the other does. Neither can move. That's deadlock. A traffic light at the bridge that alternates direction is* ***prevention****. A bridge controller who looks at incoming traffic and only allows a car when the other side is empty is* ***avoidance****. A police officer who shows up after the jam and tells one car to reverse is* ***detection & recovery****.*

6.1 The Four Necessary Conditions (Coffman conditions)

MUST KNOW HIGH FREQ AMAZON FAV

Deadlock can only occur if **all four** hold simultaneously:

1. **Mutual exclusion** — at least one resource is non-shareable (only one user at a time).
2. **Hold and wait** — a process holds at least one resource and is waiting for another.
3. **No preemption** — resources can't be forcibly taken away; only the holder releases.
4. **Circular wait** — there exists a cycle of processes, each waiting for a resource held by the next.

★ **Interview Gold:** Memorize this acronym: **MHNC** — Mutual exclusion, Hold & wait, No preemption, Circular wait. If any one of these is broken, deadlock is impossible. Prevention strategies target one of these four.

6.2 Resource Allocation Graph (RAG)

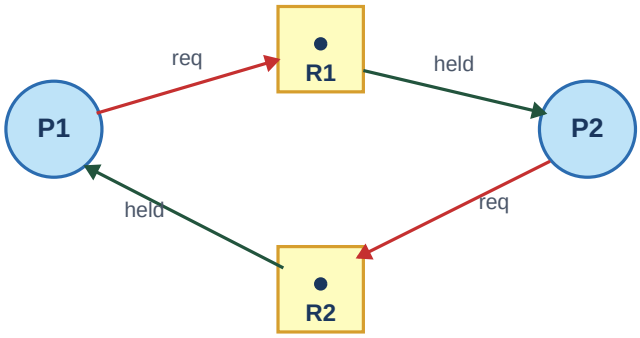
HIGH FREQ GFG CLASSIC

A directed graph where processes are circles and resources are squares. Edges:

- **Request edge**: Process → Resource (process is waiting on it).
- **Assignment edge**: Resource → Process (resource is held by it).

Single instance per resource type: a cycle in the RAG ⇔ deadlock. **Multiple instances:** a cycle is necessary but not sufficient — you need to do the Banker's-style analysis.

Resource Allocation Graph — Cycle = Deadlock



→ red: request edge

→ green: assignment edge

Cycle P1→R1→P2→R2→P1 means DEADLOCK

6.3 Deadlock Prevention — break a condition

Break	How	Trade-off
Mutual exclusion	Make resources shareable	Many resources can't be (printers, files)
Hold and wait	Request all resources upfront	Low utilization; starvation
No preemption	Forcibly take resources back	Hard for many resource types
Circular wait	Order resources; request in order	Practical & common; just needs discipline

★ **Interview Gold: Resource ordering** is the most practical prevention technique used in real code. Assign every lock a global order number; always acquire locks in increasing order. This is exactly what kernel developers do with the Linux kernel's lock ordering rules — and it kills the circular wait condition cheaply.

6.4 Deadlock Avoidance — Banker's Algorithm

HIGH FREQ NUMERICAL ALERT GFG CLASSIC

Each process declares its maximum resource needs upfront. Before granting any request, the OS simulates the allocation and asks: **is the resulting state still safe?** A state is **safe** if there exists some order in which all processes can complete with the available resources.

Banker's example

Suppose 3 resource types (A, B, C) with totals (10, 5, 7). Current allocation and max needs:

Process	Allocation (A,B,C)	Max (A,B,C)	Need = Max – Alloc
P0	0,1,0	7,5,3	7,4,3
P1	2,0,0	3,2,2	1,2,2
P2	3,0,2	9,0,2	6,0,0

P3	2,1,1	2,2,2	0,1,1
P4	0,0,2	4,3,3	4,3,1

Total allocated = (7,2,5). Available = total – allocated = (3,3,2). Find a process whose Need ≤ Available. P1's need is (1,2,2) ≤ (3,3,2) ✓. Pretend P1 finishes and releases (2,0,0): Available becomes (5,3,2). Repeat: P3 fits → (7,4,3). P0 fits → (7,5,3). P2 fits → (10,5,5). P4 fits → (10,5,7). **Safe sequence: P1 → P3 → P0 → P2 → P4.**

★ **Interview Gold:** The Banker's Algorithm is conservative — it may refuse a safe request because it can't prove safety. It requires knowing maximum needs in advance, which is impractical for general-purpose systems. In real interviews, just be able to explain the safety check and run a small example.

6.5 Deadlock Detection & Recovery

Allow deadlocks to happen, but detect them periodically. For single-instance resources: cycle detection in RAG. For multi-instance: a variant of the Banker's check (without max needs — just current allocation and request).

Recovery options:

- **Process termination**: kill one or all involved processes. Costly — they lose work.
- **Resource preemption**: take a resource from a process, roll it back. Requires rollback support (checkpoints).
- **Choose a victim** based on priority, CPU time used, etc.

6.6 The Ostrich Algorithm

Most general-purpose OSes (Linux, Windows) do **none of the above** for general deadlocks. They *ignore* the problem — bury their head in the sand — because deadlock detection is expensive and deadlocks are rare in practice. They prevent specific known cases (lock ordering in the kernel) but if your app deadlocks, you debug it.

6.7 Starvation vs Deadlock — don't confuse them

- **Deadlock**: processes are blocked forever, each waiting on the others. Nobody makes progress.
- **Starvation**: a process is repeatedly passed over and never gets the resource — but the system as a whole is making progress. Caused by unfair scheduling/priority. Fixed by aging.
- **Livelock**: processes keep changing state in response to each other but neither makes progress. Like two people in a hallway each stepping aside to let the other pass — in sync.

★ **Interview Gold: Top interview question:** *'Your program is hanging. How do you tell if it's a deadlock vs livelock vs starvation?'*
→ Deadlock: threads blocked (waiting on locks), CPU usage low for those threads.
→ Livelock: threads running, CPU usage normal/high, no useful work done.
→ Starvation: one thread blocked, others making progress.
Tools: jstack for Java, py-spy for Python, gdb thread apply all bt for C/C++.

7. Memory Management

MUST KNOW HIGH FREQ AMAZON FAV

Memory management is how the OS allocates RAM to processes, gives each process the illusion of having its own private memory, and protects them from each other. This and Virtual Memory (next chapter) are the heaviest interview topics in OS — together they account for a huge chunk of the questions.

7.1 Why memory management is hard

- Multiple processes need RAM, but RAM is finite and shared.
- Each process should think it has the whole address space (0 to MAX_VIRT_ADDR).
- Memory must be allocated, freed, and reallocated as processes start/stop.
- Processes mustn't be able to read each other's memory (protection).
- Allocation should be fast (every malloc uses it) and waste as little space as possible.

7.2 Logical vs Physical Addresses

- **Logical (virtual) address**: what your program sees. Generated by the CPU.
- **Physical address**: actual location in RAM. Sees by the memory unit.
- **Memory Management Unit (MMU)**: hardware that translates logical → physical at every access.

■ ***Analogy:*** *A logical address is like a hotel room number on your key card — Room 305. The physical address is the GPS location of that room in the world. The MMU is the hotel's lookup table that maps key cards to actual rooms. Different guests' Room 305s point to entirely different physical rooms — that's process isolation.*

7.3 Contiguous Memory Allocation

Old approach: give each process a single contiguous chunk of physical memory. Each process has a **base register** (start address) and **limit register** (size). All addresses checked against limit.

Memory is divided into **partitions**:

- **Fixed partitioning**: memory split into fixed-size blocks. Process placed in a block ≥ its size. Wastes space inside the block — **internal fragmentation**.
- **Variable partitioning**: blocks sized to fit processes exactly. Over time, free space gets split into many small unusable holes — **external fragmentation**.

7.4 Placement algorithms (for variable partitioning)

- **First Fit**: allocate the first hole big enough. Fast.
- **Best Fit**: allocate the smallest hole that fits. Leaves tiny unusable slivers; worse fragmentation in practice.
- **Worst Fit**: allocate the largest hole. Leaves big leftover holes; rarely used.
- **Next Fit**: like first fit but starts searching from last allocation. Spreads allocations around.

★ **Interview Gold:** Counterintuitive result: **First Fit is usually better than Best Fit in practice**, both in speed and fragmentation. Best Fit creates many tiny holes that no future request can use.

7.5 Fragmentation

• **Internal fragmentation**: allocated space > needed space. Happens in fixed-size blocks (or in pages). Wasted space is *inside* an allocated unit.

• **External fragmentation**: total free memory is enough, but it's broken into many non-contiguous holes. No single hole is big enough for the new request.

■ ***Analogy:*** *Internal fragmentation: you booked a hotel suite for 1 person — 3 beds wasted in your room. External fragmentation: you want a 5-bed family room, the hotel has 3 single rooms and 2 doubles scattered across floors — 7 beds total free, but no single room with 5.*

Solutions to external fragmentation: Compaction (shuffle allocations to merge free holes — slow), or **paging/segmentation** (allow non-contiguous allocation).

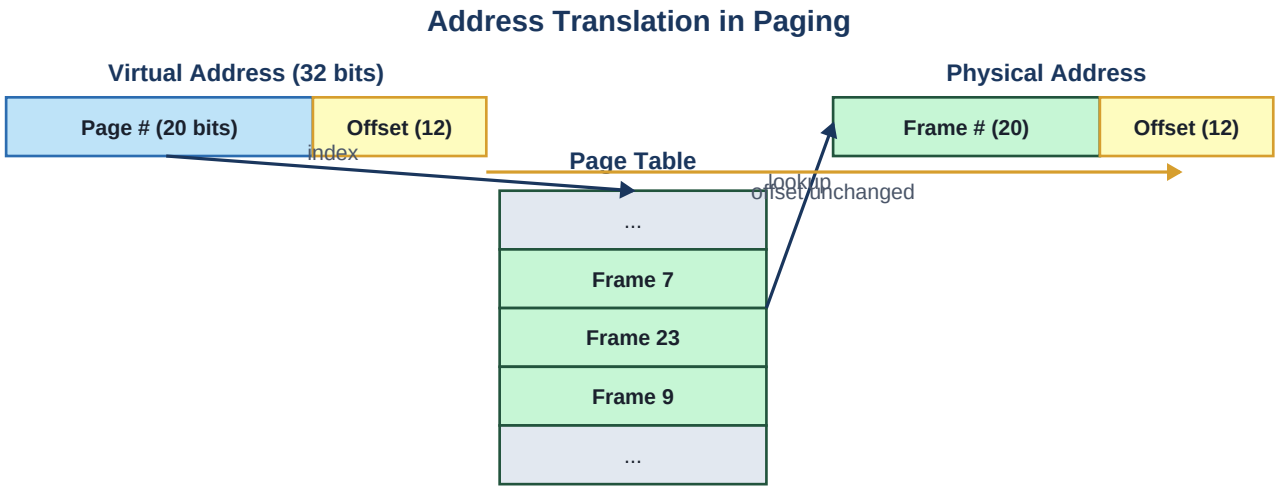
7.6 Paging — the modern standard

MUST KNOW HIGH FREQ AMAZON FAV GFG CLASSIC

Divide physical memory into fixed-size **frames** and logical memory into same-size **pages** (typically 4 KB). A **page table** per process maps page numbers to frame numbers. Pages of one process can live in any frames — non-contiguous physically, contiguous logically. No external fragmentation. Some internal fragmentation in the last page of each process.

Address translation

A logical address of *m* bits is split into: top bits = page number *p*, bottom bits = offset *d*. The MMU looks up *p* in the page table → gets frame number *f*. Physical address = (f << offset_bits) | d.



Page # is replaced by Frame # from the table; offset stays the same.

```
Logical address (32 bits, 4 KB pages):
[ 20-bit page number | 12-bit offset ]
                |
                v  (page table lookup)
[ 20-bit frame number | 12-bit offset ]
Physical address
```

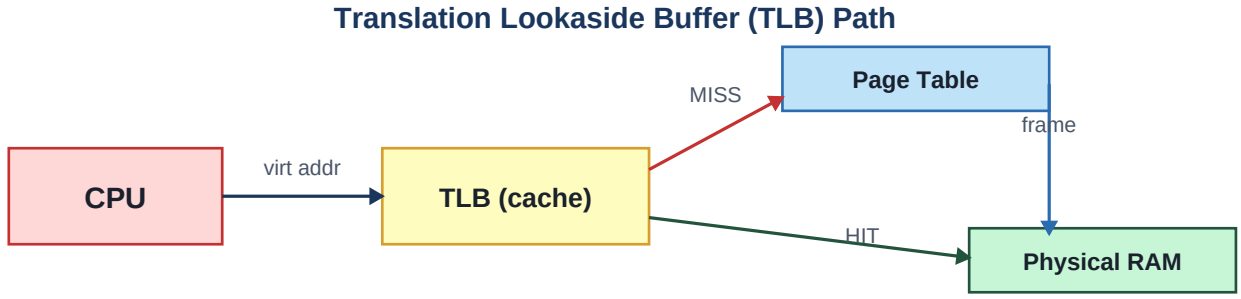
Numerical: addresses you might be asked

- 32-bit logical address, 4 KB page size → page number = 20 bits, offset = 12 bits, 2^{20} = ~1M page table entries.
- Each PTE is typically 4 bytes → page table size = 4 MB *per process*. Big!
- This is why we have **multi-level page tables** and **inverted page tables** — to avoid the storage explosion.

7.7 Translation Lookaside Buffer (TLB)

MUST KNOW NUMERICAL ALERT GFG CLASSIC

Looking up the page table for every memory access would be brutally slow (each access becomes two: one for the PTE, one for the data). The **TLB** is a small, fast hardware cache of recently used page table entries. Typical TLB has 64–1024 entries.



TLB hit avoids the extra memory access for the page table.

Effective Access Time (EAT) when TLB hit rate is *h*, TLB time *t*, memory time *m*:
EAT = h(t + m) + (1 – h)(t + 2m)

★ **Interview Gold:** Classic numerical: hit rate 80%, TLB access 20ns, memory 100ns.
EAT = 0.8 × (20 + 100) + 0.2 × (20 + 200) = 0.8 × 120 + 0.2 × 220 = 96 + 44 = **140 ns**. Without TLB it would be 200 ns. TLB pays for itself massively.

7.8 Multi-level paging

Instead of one giant page table, use a hierarchy. The top-level table indexes into many smaller second-level tables, only some of which exist. Saves memory when address spaces are sparse.

x86-64 uses 4 levels (PML4 → PDPT → PD → PT). Each level adds one memory access on TLB miss, but TLB makes the common case fast.

7.9 Segmentation

Memory divided by **logical units** (code, data, stack, heap), not fixed-size blocks. Each segment has a base and limit. Addresses are (segment, offset). Matches programmer's mental model.

Pros: Natural protection (e.g., code segment = read-only). **Cons:** External fragmentation returns.

7.10 Paged Segmentation

Combines both: divide into segments (logical view), but each segment is paged (physical implementation). x86 historically used segmentation + paging; x86-64 essentially flattened segments and relies on paging.

7.11 Common interview questions

- **Paging vs Segmentation:** Paging uses fixed-size pages (no external frag, but internal frag); segmentation uses variable-size segments matching logical units (external frag, no internal frag).
- **Why is page size 4 KB?** Trade-off: small pages → big page tables. Large pages → more internal fragmentation. 4 KB has been the sweet spot historically; modern systems also support 'huge pages' (2 MB / 1 GB) for large workloads.
- **What's a page fault?** When the CPU accesses a page that isn't in memory (or has no valid PTE). Triggers a trap to the OS. *Covered in detail in the next chapter.*
- **How does the OS protect process A's memory from process B?** Each process has its own page table. On a context switch, page tables are swapped (CR3 register on x86). A can only address frames that B's page table maps — and they don't overlap (except for explicitly shared mappings).

8. Virtual Memory

MUST KNOW HIGH FREQ AMAZON FAV NUMERICAL ALERT

Virtual memory is the OS trick that lets you run programs bigger than physical RAM by keeping only the actively used pages in memory and the rest on disk. It's also what gives each process its own isolated address space. Together with paging, this is the single most-asked OS topic in interviews.

■ ***Analogy:*** *Your desk (RAM) is small. Your filing cabinet (disk) is huge. You only keep on your desk the documents you're currently using. When you need a paper that's in the cabinet, you swap one off your desk back to the cabinet to make room. To the program (you), it feels like you have infinite desk space — you just sometimes wait a moment for paper to arrive. That's virtual memory.*

8.1 Demand Paging

Don't load all pages of a program when it starts. Instead, load each page **on demand** — the first time it's accessed. This is lazy loading at the OS level.

How it works:

1. Each PTE has a **valid bit**. Initially most PTEs are invalid (page not in memory).
2. CPU accesses a page → MMU sees invalid PTE → raises **page fault**.
3. OS handler runs: finds the page on disk, picks a frame (maybe evicting another page), loads the page, updates the PTE, returns to user code.
4. The faulting instruction is re-executed, and now succeeds.

8.2 Page Fault — the cost

A page fault is **expensive**. A normal memory access is ~100 ns. A page fault to disk is ~10 ms (HDD) or ~100 µs (SSD) — that's 6 orders of magnitude slower than RAM.

Effective Access Time with page faults:
EAT = (1 – p) × m + p × page_fault_time
where *p* = page fault probability, *m* = memory access time.

★ **Interview Gold:** Numerical: p = 0.001 (one in 1000), m = 100 ns, page_fault_time = 10 ms = 10,000,000 ns.
EAT = 0.999 × 100 + 0.001 × 10,000,000 = 99.9 + 10,000 ≈ **10,100 ns**.
Even a 0.1% page fault rate makes memory 100× slower on average. That's why minimizing page faults is the entire point of cache-friendly programming.

8.3 Page Replacement Algorithms

MUST KNOW HIGH FREQ NUMERICAL ALERT GFG CLASSIC

When memory is full and a new page is needed, which page do we evict? This is the central question of virtual memory. Algorithms compared by **number of page faults** for a given reference string.

FIFO — First In First Out

Evict the page that's been in memory longest. Simple. Suffers from **Belady's anomaly**.

★ **Interview Gold: Belady's anomaly**: in FIFO, increasing the number of frames can *increase* the number of page faults. Counterintuitive — interviewers love this. Classic example: reference string 1,2,3,4,1,2,5,1,2,3,4,5 has more faults with 4 frames than 3 frames in FIFO.

Optimal (OPT / MIN) — Belady's algorithm

Evict the page that won't be used for the longest time in the future. **Provably optimal** (minimum page faults). Cannot be implemented (requires future knowledge) — used as a benchmark to compare others.

LRU — Least Recently Used

Evict the page that hasn't been used for the longest time. Approximates OPT because past behavior often predicts future behavior. Best practical algorithm but expensive to implement exactly (must timestamp every access or maintain a doubly-linked list).

Approximations: **Clock algorithm** (Second Chance) — circular list of pages with a reference bit. Sweep through; if ref bit = 1, clear it and continue. If ref bit = 0, evict. Cheap and used in real OSes.

LFU — Least Frequently Used

Evict the page accessed fewest times. Hurts pages that were heavily used recently and now aren't.

8.4 Page Replacement Numerical Template

NUMERICAL ALERT MUST KNOW

Given a reference string and N frames, simulate. Frames start empty (each fill = page fault). Use a table tracking frame contents at each step.

Page Replacement (LRU, 3 frames) — Reference: 7 0 1 2 0 3 0 4

Ref	7	0	1	2	0	3	0	4
F1	7	7	7	2	2	2	2	4
F2	-	0	0	0	0	0	0	0
F3	-	-	1	1	1	3	3	3
Flt	Y	Y	Y	Y	N	Y	N	Y

Total page faults = 6 (red Y cells)

Example (LRU, 3 frames, reference: 7,0,1,2,0,3,0,4,2,3,0,3,2)

Ref	F1	F2	F3	Fault?
7	7	-	-	Y
0	7	0	-	Y
1	7	0	1	Y
2	2	0	1	Y (evict 7, LRU)

0	2	0	1	N
3	2	0	3	Y (evict 1)
0	2	0	3	N
4	4	0	3	Y (evict 2)
2	4	0	2	Y (evict 3)
3	4	3	2	Y (evict 0)
0	0	3	2	Y (evict 4)
3	0	3	2	N
2	0	3	2	N

Total page faults = **9**. Memorize the technique — almost every fresher OS interview has one of these.

8.5 Thrashing

HIGH FREQ MUST KNOW

When a process (or the system) spends more time paging than executing, you have **thrashing**. Throughput collapses; the CPU is mostly idle waiting for disk.

Cause: too many processes, each with a working set larger than its allotted frames. Each page fault evicts a page another process needs, causing a cascade.

■ ***Analogy:*** *Thrashing is trying to study for 5 exams at once with a tiny desk. You spend all your time swapping books between the desk and the shelf, and zero time actually reading. The fix is studying fewer subjects at a time (reduce multiprogramming level).*

8.6 Working Set Model

The **working set** of a process at time *t* is the set of pages referenced in the last Δ accesses. If the OS keeps each process's working set fully resident in memory, that process won't thrash. If total working set size > physical memory, the OS swaps out entire processes (not pages) until it fits.

★ **Interview Gold: Locality of reference** is the principle that makes virtual memory work. Programs don't access memory randomly — they access nearby addresses (spatial locality) and re-access the same locations (temporal locality). If they accessed memory randomly, every access would page-fault and computers would be unusable. The whole memory hierarchy (registers → L1 → L2 → L3 → RAM → disk) bets on locality.

8.7 Copy-on-Write (COW)

HIGH FREQ AMAZON FAV

When fork() is called, instead of duplicating all of the parent's pages, parent and child share the same physical pages — but marked read-only. The first time either tries to write, a fault occurs and the OS makes a private copy. Saves massive memory if the child immediately execs (very common pattern: fork-then-exec for shell commands).

8.8 Memory-mapped files (mmap)

mmap() maps a file directly into the process's address space. Reading the memory IS reading the file (loaded on demand via page faults). Faster than read() for big files, and lets multiple processes share the same mapped file as IPC. Most databases use mmap internally.

8.9 Common interview questions

- **How does Linux's OOM killer decide which process to kill?** It scores processes by memory usage, fork count, niceness, etc. The one with the highest *oom_score* dies. Critical services have low scores configured.
- **What's the difference between paging and swapping?** Paging moves *individual pages* in/out. Swapping (classical) moves *entire process* images. Modern systems use paging exclusively, but the term 'swap space' (the disk area used) survives.
- **What's a 'major' vs 'minor' page fault?** Minor: PTE was invalid but the page is already in memory (e.g., shared with another process). Fix in microseconds. Major: page must be read from disk. Milliseconds.
- **Why are page sizes a power of 2?** So the address can be split into page-number and offset by simple bit shifting — no division needed.

9. File Systems

HIGH FREQ SERVICE-BASED FAV

A **file system** is the OS's way of organizing data on storage. To the user it looks like a tree of folders and files; under the hood it's a structure of blocks, metadata, and free-space lists carved out of a flat array of disk sectors. This chapter is usually lighter in interviews than scheduling or memory, but inodes and allocation strategies do come up.

9.1 File concepts

- **File** — a named sequence of bytes, with metadata (size, permissions, timestamps).
- **Directory** — a special file that maps names to file metadata.
- **Mount point** — where another file system's root is attached into the existing tree.
- **File descriptor (fd)** — small integer the OS gives you after open(); index into a per-process file table.

■ ***Analogy:*** *A file system is a library. Files are books. Directories are shelves (and sections). The card catalog (inodes) tells you which physical shelf and slot a book is on, plus author, page count, etc. When you ask for 'Harry Potter / Chapter 3', the librarian looks it up in the catalog, finds shelf 4B-slot 12, and fetches it. The file descriptor is your library check-out slip — a small ID that lets the librarian track what you've got out without you carrying the whole catalog around.*

9.2 File operations & access methods

Common operations: create, open, read, write, seek, close, delete.

Access methods:

- **Sequential** — read/write in order. Like a tape. Fast for streaming.
- **Direct (random)** — jump to any offset with seek. What databases need.
- **Indexed** — separate index file maps keys → offsets. Like a book's table of contents.

9.3 Directory Structure

- **Single-level**: one big flat directory. Name conflicts; doesn't scale.
- **Two-level**: one directory per user. Better, but limited.
- **Tree-structured**: directories can contain subdirectories. The standard.
- **Acyclic graph**: support hard links / shared files — multiple names for one file.
- **General graph**: allows cycles (via symbolic links). Needs care during traversal.

9.4 File Allocation Methods

How are a file's blocks laid out on disk?

Contiguous allocation

All blocks of a file are stored contiguously. Like reading a magazine — flip pages, you're there. **Pros:** excellent sequential and random access. **Cons:** external fragmentation; files can't grow easily.

Linked allocation

Each block contains a pointer to the next. Like a linked list. **Pros:** no external fragmentation; files grow easily. **Cons:** random access is O(n) — must walk the list. A single block corruption loses everything after it.

Indexed allocation

Each file has an **index block** containing pointers to all its data blocks. **Pros:** O(1) random access, no external fragmentation. **Cons:** overhead for small files (a whole block just for the index). Index block size limits file size.

Variations: linked indexes (last entry of index block points to next index block), multi-level index (index of indexes), and combined schemes (used by Unix inodes — next section).

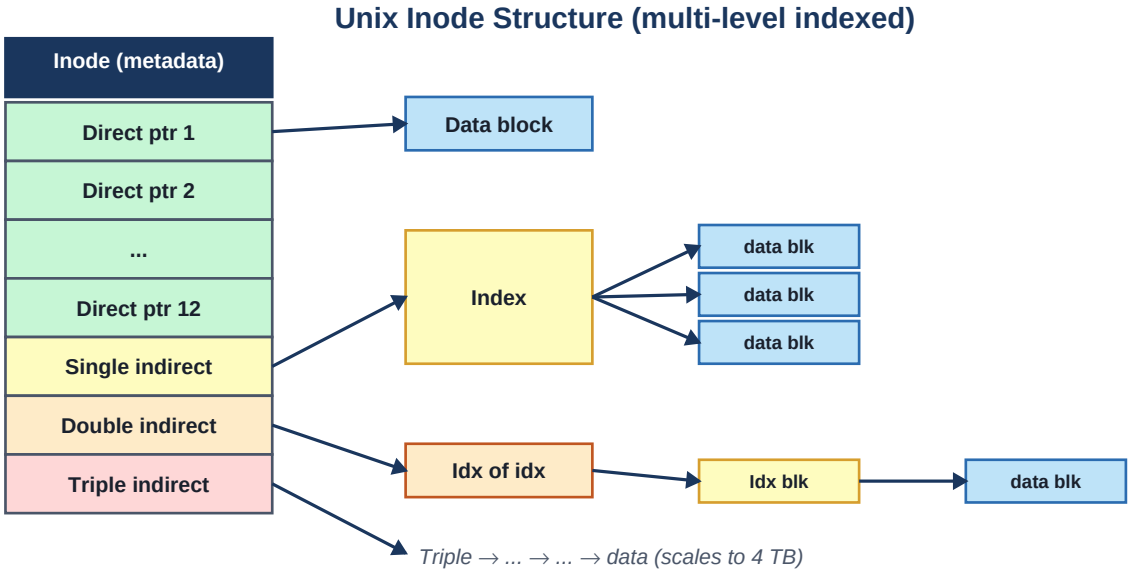
9.5 The Unix inode

MUST KNOW HIGH FREQ GFG CLASSIC

A masterpiece of file system design. An **inode** is the metadata structure for a file — owner, permissions, size, timestamps, and pointers to data blocks. Filename is NOT in the inode; it's in the containing directory.

Each inode contains a fixed number of block pointers, structured as:

- ~12 **direct** pointers — point straight to data blocks. Handle small files efficiently.
- 1 **single indirect** pointer — points to a block of data block pointers.
- 1 **double indirect** pointer — points to a block of single-indirect pointers.
- 1 **triple indirect** pointer — points to a block of double-indirect pointers.



More indirection = larger files, with fixed inode size.

With 4KB blocks and 4-byte pointers (1024 pointers per indirect block): direct = 48 KB, single = 4 MB, double = 4 GB, triple = 4 TB. The structure scales gracefully from tiny files to huge files without the overhead of always allocating big index structures.

★ **Interview Gold: Why is the filename not in the inode?** Because that enables **hard links**: multiple directory entries can point to the same inode. The file is only deleted when its link count drops to zero. Symbolic (soft) links are different — they're separate files containing a path.

9.6 Hard Links vs Soft Links

HIGH FREQ AMAZON FAV GFG CLASSIC

Aspect	Hard Link	Soft Link (Symlink)
What it is	Extra directory entry to same inode	Separate file containing a path string
Cross filesystems?	No (inodes are local)	Yes
Survives target delete?	Yes — file alive until last link gone	Becomes a 'dangling' link
For directories?	Usually disallowed (cycles)	Allowed
Size	Negligible	Stores the path (a few bytes)

9.7 Free Space Management

- **Bitmap**: one bit per block (0 = free, 1 = used). Fast to find free blocks; easy to find contiguous runs.
- **Linked list**: free blocks linked together. Easy to maintain, but finding contiguous runs is hard.
- **Grouping**: first free block stores addresses of next N free blocks, then chains.

9.8 Journaling

What happens if the system crashes mid-write? You can end up with metadata pointing to data that was never written, or vice versa. **Journaling** file systems (ext4, NTFS, XFS) first write the intended changes to a **journal** (log), then apply them. After a crash, the OS replays the journal — uncompleted operations are rolled back or forward to a consistent state.

■ ***Analogy:*** *Journaling is like writing what you're about to do on a sticky note before doing it. If you get interrupted, the sticky note tells you (or the next person) what was in progress and how to finish or undo it cleanly. Without it, a power cut mid-rename could leave you with two directory entries pointing to garbage.*

9.9 Common File Systems

FS	Used on	Notable for
ext4	Linux	Journaling, extents, large file support
NTFS	Windows	Journaling, ACLs, compression, encryption
FAT32 / exFAT	USB drives, SD cards	Simple, universal compatibility
APFS	macOS / iOS	Copy-on-write, snapshots, encryption

ZFS / Btrfs	Servers, NAS	Snapshots, checksums, software RAID

9.10 Common interview questions

• **What's the difference between rm and unlink?** unlink is the syscall that removes a directory entry. rm is the user command that calls unlink (for files) or rmdir (for directories).

• **If 5 processes have the same file open and one calls unlink, is the file gone?** The directory entry is removed (other names lose access), but the inode is kept alive as long as there are open file descriptors. The space is freed only when the last fd is closed. This is the basis for the Unix idiom of 'create temp file, unlink immediately, keep using via fd'.

• **What is a VFS?** Virtual File System — the kernel's abstraction layer that lets multiple FS types (ext4, NTFS, NFS) live behind one syscall interface.

• **What is mmap-based file I/O vs read/write?** mmap maps file pages into memory; you access the file like an array. read/write copies through kernel buffers. mmap is faster for random access and big files, especially with multiple processes sharing.

10. I/O & Disk Scheduling

HIGH FREQ NUMERICAL ALERT GFG CLASSIC

I/O is the slowest part of any computer. Disk scheduling is about ordering disk requests so the physical head moves the least distance possible — a classic optimization problem with several named algorithms. Expect at least one disk scheduling numerical in interviews.

10.1 How I/O works (10000-foot view)

- Device controller has its own registers and (often) buffer memory.
- **Polling**: CPU repeatedly checks the device status. Wastes CPU.
- **Interrupts**: device signals the CPU when ready. Standard approach.
- **DMA (Direct Memory Access)**: dedicated chip transfers data between device and RAM without involving CPU. CPU is interrupted only on completion.

■ ***Analogy:*** *Polling is calling the pizza place every 30 seconds asking 'is it ready yet?'. Interrupts are giving them your phone number and waiting for their call. DMA is hiring a delivery service that picks up the pizza and brings it to your fridge while you nap — you only wake up when it rings the doorbell.*

10.2 Disk geometry (HDD)

An HDD has stacked **platters**, each with two surfaces. Each surface has concentric **tracks**, and each track is divided into **sectors** (typically 512 B or 4 KB). The **cylinder** is the set of tracks at the same radius across all surfaces. A **read/write head** per surface moves together on an actuator arm.

Access time for a disk read = seek time + rotational latency + transfer time.

- **Seek time**: move the head to the right cylinder. Dominant cost (~5–10 ms).
- **Rotational latency**: wait for the right sector to spin under the head. Average = half a revolution. At 7200 RPM, that's ~4 ms.
- **Transfer time**: actually read the data. Microseconds for a sector.

★ **Interview Gold:** Disk scheduling is all about **minimizing seek time**. Rotational latency and transfer are basically fixed; seek is the variable we can optimize by reordering requests.

10.3 Disk Scheduling Algorithms

FCFS (First-Come, First-Served)

Process requests in arrival order. Simple, fair, but causes lots of head movement if requests are scattered.

SSTF (Shortest Seek Time First)

Pick the request closest to the current head position. Reduces total seek significantly. Can starve far requests.

SCAN (Elevator Algorithm)

Head moves in one direction, servicing all requests on the way, then reverses at the end. Like an elevator picking up everyone going up, then turning around.

C-SCAN (Circular SCAN)

Like SCAN, but on reaching the end, head jumps back to the start (without servicing) and scans in the same direction. Fairer than SCAN (uniform wait time).

LOOK and C-LOOK

Like SCAN/C-SCAN but the head reverses (or jumps) at the last request in that direction, not all the way to the disk end.

10.4 Disk Scheduling Numerical

NUMERICAL ALERT MUST KNOW GFG CLASSIC

Setup: Disk has cylinders 0–199. Head currently at 53. Queue of pending requests (in arrival order): **98, 183, 37, 122, 14, 124, 65, 67**. Compute total head movement for each algorithm.

Algorithm	Service order	Total movement
FCFS	53 → 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67	640
SSTF	53 → 65 → 67 → 37 → 14 → 98 → 122 → 124 → 183	236
SCAN (toward 199)	53 → 65 → 67 → 98 → 122 → 124 → 183 → 199 → 37 → 14	208
C-SCAN (toward 199)	53 → 65 → 67 → 98 → 122 → 124 → 183 → 199 → 0 → 14 → 37	382
LOOK (toward 199)	53 → 65 → 67 → 98 → 122 → 124 → 183 → 37 → 14	299
C-LOOK (toward 199)	53 → 65 → 67 → 98 → 122 → 124 → 183 → 14 → 37	322

★ **Interview Gold: Quick computation tip:** for SCAN-family, list pending cylinders in sorted order, locate the current head, and trace movement based on direction. Total movement = sum of absolute jumps. For C-SCAN/C-LOOK, the jump back doesn't service anything but counts toward total movement on standard problems (check the convention your interviewer uses — some count only serviced movements).

10.5 SSDs change the game

SSDs have no moving parts. Access time is roughly the same for any location — **seek time ≈ 0**. Classical disk scheduling algorithms still apply but the gains are tiny. SSD-specific concerns:

- **Wear leveling**: each flash cell can only be erased ~3000–100,000 times. The controller spreads writes evenly across cells.
- **Garbage collection & TRIM**: deleted data isn't immediately erased; OS sends TRIM to tell the SSD which blocks can be reclaimed.
- **Read/write asymmetry**: reads are fast and per-page. Writes require erasing whole blocks (much larger). This is the source of write amplification.

10.6 Spooling

Spooling (Simultaneous Peripheral Operations On-Line): buffer slow I/O (like printing) to disk so the slow device can be fed from a queue at its own pace, freeing up programs immediately. Print spoolers, mail queues are classic examples.

10.7 Common interview questions

- **What's the difference between blocking and non-blocking I/O?** Blocking: read() doesn't return until data is available. Non-blocking: returns immediately with whatever's available (possibly nothing, with EAGAIN). Non-blocking is used with select/poll/epoll for handling many connections in one thread.
- **What is epoll?** Linux's scalable I/O event notification. select/poll are O(n) per call; epoll is O(1) for monitoring + O(events) for processing. Powers Nginx, Node.js, etc.
- **Why is async I/O important?** Lets a single thread handle thousands of concurrent connections without spawning thousands of threads. The reactor pattern (Node, Netty) is built on this.
- **What is double buffering?** Use two buffers — fill one while the device reads the other. Hides I/O latency. Video, audio, and graphics pipelines use this universally.

11. System Calls, Signals & OS-Level Patterns

HIGH FREQ SERVICE-BASED FAV

This bonus chapter covers topics that don't quite fit anywhere else but show up in interviews and real-world code: system calls in detail, signals, the OS view of networking, and how high-level constructs (Python's GIL, Node's event loop, Go's goroutines) sit on top of these OS primitives.

11.1 System Calls — the boundary between you and the kernel

MUST KNOW AMAZON FAV

Every interesting thing your program does — reading a file, allocating memory, opening a socket, creating a process — ultimately becomes a system call. Understanding the syscall boundary is the single best way to look like a thoughtful engineer in an interview.

How a system call actually works

1. Your code calls a libc wrapper like read(fd, buf, n).
2. The wrapper puts the syscall number into a register and arguments into other registers.
3. It executes a special instruction (syscall on x86-64, svc on ARM).
4. CPU switches to kernel mode, jumps to a fixed entry point.
5. Kernel dispatches to the handler based on the syscall number.
6. Handler runs; result is placed in a register.
7. sysret / eret returns to user mode, back to your code.

★ **Interview Gold:** A system call is **two mode switches and several memory accesses** — typically a few hundred nanoseconds even when it does nothing. That's why batching syscalls (e.g., writev instead of many writes, sendfile instead of read + write) is a real performance technique.

System call categories

Category	Examples	What they do
Process control	fork, exec, exit, wait, kill	Create, terminate, control processes
File management	open, read, write, close, lseek, stat	Talk to the file system
Device management	ioctl, read, write	Direct device control
Information	getpid, gettimeofday, uname	Query system state
Communication	pipe, socket, send, recv, mmap, shmget	IPC and networking
Protection	chmod, chown, umask, setuid	Permissions and identity

11.2 Signals — async notifications to processes

HIGH FREQ GFG CLASSIC

A **signal** is a small integer the kernel (or another process) sends to a process to notify it of an event. Receiving a signal interrupts whatever the process is doing. Each signal has a default action (usually: terminate). You can install a custom handler with sigaction().

Signal	What it means	Default action
SIGINT	Ctrl-C from terminal	Terminate
SIGTERM	Polite request to terminate	Terminate
SIGKILL	Force kill (cannot be caught!)	Terminate immediately
SIGSTOP	Pause process (uncatchable)	Stop
SIGCONT	Continue a stopped process	Continue
SIGSEGV	Segmentation fault (invalid memory)	Terminate + core dump
SIGPIPE	Write to a pipe with no reader	Terminate
SIGCHLD	Child process exited	Ignore
SIGALRM	Timer expired	Terminate

★ **Interview Gold: SIGKILL and SIGSTOP cannot be caught, blocked, or ignored.** This is the kernel's escape hatch — without these, a misbehaving process could refuse to die. kill -9 PID sends SIGKILL. If even that doesn't kill the process, the process is stuck in an uninterruptible kernel state (usually waiting on broken I/O) — and you need to reboot.

■ ***Analogy:*** *Signals are like a fire alarm in a building. Most alarms can be silenced or ignored if you have a key (signal handler). SIGKILL is the fire marshal kicking the door down — no one can stop it. SIGSTOP is the building manager locking the elevators until further notice.*

11.3 Network I/O at the OS level

Sockets are the OS abstraction for network endpoints. Quick mental model of a TCP connection:

1. Server: socket() → bind(port) → listen() → accept().
2. Client: socket() → connect(addr).
3. Both: send() / recv() to transfer data.
4. Both: close() when done.

Each connection uses a file descriptor. To handle 10,000 concurrent connections, naive approaches either spawn 10,000 threads (death by context switching) or use a single-threaded event loop with non-blocking I/O and epoll/kqueue/IOCP to multiplex events.

★ **Interview Gold:** The **C10K problem** (handling 10K concurrent connections) drove modern server design. Solutions: non-blocking I/O + event loop (Nginx, Node.js), or M:N user-space scheduling (Go, Erlang). Both leverage the OS giving you tools (epoll, edge-triggered notifications) rather than the OS doing the work for you.

11.4 Where OS meets your runtime

Python's GIL

The **Global Interpreter Lock** is a mutex inside CPython that ensures only one thread executes Python bytecode at a time. The OS schedules Python threads as normal kernel threads, but the GIL serializes them at the interpreter level. Result: Python threads don't give you CPU parallelism for Python code — for that, use multiprocessing (separate processes, separate GILs) or move the hot code to C extensions that release the GIL.

Node.js's event loop

Single-threaded JavaScript on top of an event loop. The loop polls for completed I/O events (via epoll/kqueue) and runs registered callbacks. CPU-bound work blocks the loop — that's why Node is great for I/O-heavy services but bad for compute. Workers run in a libuv thread pool for things like file I/O and crypto.

Go's goroutines

Goroutines are M:N scheduled by the Go runtime onto a small pool of kernel threads. A goroutine costs ~2 KB of stack initially (vs ~2 MB for a kernel thread). Millions of goroutines are feasible. When a goroutine blocks on I/O, the runtime moves another goroutine onto the kernel thread — to the kernel, the thread is always busy.

Java's threads

Java threads are 1:1 with OS threads (since the late '90s). Concurrency primitives like synchronized and java.util.concurrent.locks map down to OS mutexes/futexes. Modern Java (21+) adds **virtual threads** — M:N-scheduled green threads that bring goroutine-like cheapness.

★ Interview Gold: Every high-level concurrency model eventually maps to OS-level primitives. Goroutines, virtual threads, coroutines, async/await — under the hood, they're all 'park this user-level task and let the OS thread do something else, then resume when ready'. Knowing this gives you the vocabulary to debate trade-offs in interviews.

11.5 Real-world OS-level performance tips

- **Minimize syscalls.** Each one is a mode switch. Buffer I/O; batch where possible.
- **Pin threads to cores** for low-latency systems — avoids cache thrashing on migration.
- **Use huge pages** (2 MB / 1 GB) when working with large datasets — fewer TLB misses.
- **Avoid unnecessary copies.** sendfile, splice, zero-copy networking all matter at scale.
- **Watch your page faults.** A 1% page fault rate can make memory access 100× slower on average.
- **Profile, don't guess.** Tools: perf, strace, ltrace, htop, iostat, vmstat, flamegraphs.

11.6 OS & Security — quick notes

- **ASLR (Address Space Layout Randomization)**: randomizes stack, heap, library addresses on each run. Makes buffer-overflow exploits harder.
- **DEP / W^X**: pages are either writable OR executable, never both. Stops classic shellcode injection.
- **Stack canaries**: a random value placed before the return address. Function epilogue checks it; if changed, abort. Detects stack smashing.
- **Sandboxing**: restrict what syscalls a process can make (e.g., seccomp on Linux). Used by browsers, container runtimes.

• **Capabilities**: fine-grained permission bits replacing the all-or-nothing root model (e.g., a process that only needs to bind low ports can get CAP_NET_BIND_SERVICE without full root).

Containers, cgroups, and the "modern" deployment topics are covered in depth in the next chapter.

12. Modern Systems Patterns

Classical OS theory ends at chapter 11. But modern interviews — especially at companies that operate at scale — go further. They want to know whether you understand how today's runtimes, containers, databases, and high-performance services exploit OS primitives. This chapter is the bridge from textbook to production. Reading it well separates you from 90% of freshers.

★ **Interview Gold:** Even when these topics are framed as "system design", the answers live in OS fundamentals. Knowing why event loops beat thread pools, or how Docker reuses image layers, is just applied OS knowledge.

12.1 Why Async Beats Threads at Scale (The C10K Problem)

MUST KNOW SERVICE-BASED FAV AMAZON FAV

Imagine you're building a chat server expecting 100,000 concurrent users. Each user sits idle most of the time, occasionally sending a message. How do you design the server?

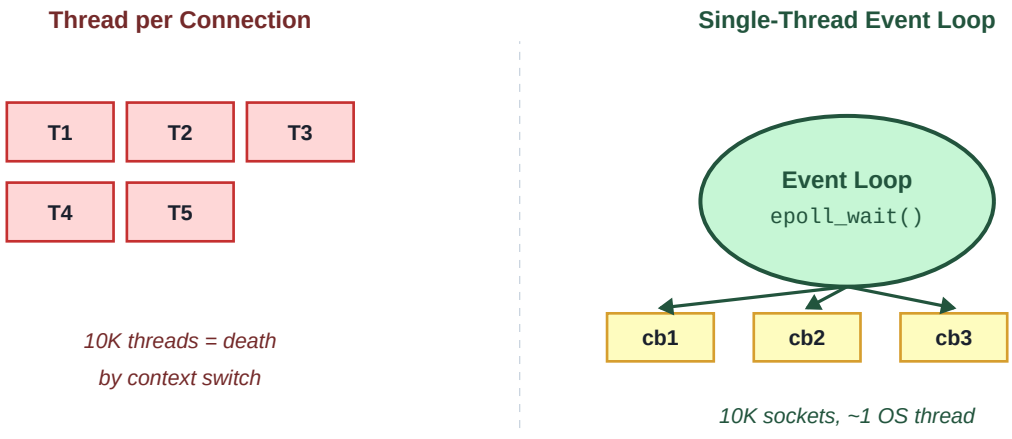
Approach 1: Thread per connection

Spawn one OS thread per connected client. Simple. Each thread blocks in recv() until a message arrives. **Problem:** 100K threads = ~200 GB of stack memory (default 2 MB stack each), plus context-switch overhead crushes the CPU. The OS scheduler is now your bottleneck. This is the **C10K problem** — the wall the industry hit in the early 2000s.

Approach 2: Event loop + non-blocking I/O

One thread. Register all 100K sockets with the kernel via epoll (Linux), kqueue (BSD/macOS), or IOCP (Windows). Call epoll_wait() — it blocks until *any* of the registered sockets becomes ready, then returns just those. Handle their callbacks. Loop. **Total OS threads: 1.** Total memory: kilobytes per connection.

Event Loop vs Thread-per-Connection (the C10K answer)



■ ***Analogy:*** *Thread-per-connection is hiring 100,000 receptionists, one per phone, mostly sitting silent. Event loop is one ridiculously fast receptionist with a switchboard — the lightbulb on a line lights up when a call needs attention, they handle it in 200 microseconds, and move on. The switchboard (epoll) is what makes the model viable.*

★ **Interview Gold: The interview punchline:** async wins because *blocking I/O is the bottleneck, not CPU*. When 99% of your threads are blocked on I/O, paying the cost of an OS thread for each is pure waste. The event loop replaces N blocked threads with N callbacks queued on 1 thread.

Why epoll specifically?

The older select() and poll() calls require you to pass ALL watched file descriptors on every call, and the kernel scans the whole list each time — O(N) per call. epoll keeps the watched set in the kernel and only returns ready events — O(events). For 10K idle sockets with 10 active, select() does 10,000 work; epoll does 10. That's why Nginx, Node, Redis all use epoll.

12.2 How Node.js Handles Thousands of Sockets

HIGH FREQ SERVICE-BASED FAV

Node.js is the canonical event-loop runtime. Its design is layered:

1. V8 executes your JavaScript on one main thread.

2. libuv is the C library underneath — it owns the event loop and an internal thread pool (default 4 threads) for ops the OS can't do non-blocking (filesystem, DNS, crypto).

3. Your fs.readFile(cb) doesn't block JS — libuv dispatches it to a worker thread, then queues the callback when done.

4. net.createServer() uses epoll/kqueue directly; sockets are inherently non-blocking, no thread pool needed.

★ **Interview Gold: Node.js falls over on CPU-bound work.** A single while(true){} in any callback blocks the entire process — no other request gets served. That's why Node is great for chat servers, API gateways, and proxies (mostly I/O), but bad for image processing or ML inference (compute-bound). For CPU work, use worker_threads or split the service.

12.3 How the Go Scheduler Works (High Level)

RARE BUT IMPRESSIVE SERVICE-BASED FAV

Go solves the C10K problem differently. Instead of forcing you to write callbacks, it lets you write code that looks synchronous (conn.Read(buf)) but multiplexes thousands of goroutines onto a small pool of OS threads. This is the **M:N scheduling model**: M goroutines onto N OS threads.

Go's scheduler is often described by three letters:

- **G** — Goroutine. A user-space thread, ~2 KB initial stack, grows on demand.
- **M** — Machine. An OS thread. The actual unit the kernel schedules.
- **P** — Processor. A logical context that holds a queue of runnable Gs. Number of Ps = GOMAXPROCS (defaults to number of cores).

A G runs on an M, and an M can only run G's from the P it's currently attached to. When a G makes a blocking syscall, the M is parked with that G, and the P is detached and given to another M — so the remaining goroutines keep running. When a G blocks on a network operation, the runtime uses epoll/kqueue under the hood and parks the G in user space, freeing the M immediately.

■ ***Analogy:*** *G's are passengers, M's are buses, P's are routes. Normally each route (P) has one bus (M) cycling through it, picking up passengers (Gs). If a passenger asks the driver to wait at a stop indefinitely (blocking syscall), the bus stays — but the route gets a replacement bus to keep moving*

everyone else. Network I/O is the magic: the bus driver just notes 'wake this passenger when their text comes' and keeps driving.

★ **Interview Gold:** Why is Go's model often faster than Node's for I/O AND CPU? Because goroutines give you cheap concurrency (like Node) AND can actually use multiple cores (unlike Node, which needs worker_threads). The trade-off is that Go's runtime is heavier and the language is more strict.

12.4 JVM Memory Regions

HIGH FREQ MUST KNOW

When you run a Java program, the JVM carves the process's heap into several distinct regions. Knowing these is essential for interviews involving Java, and even for non-Java backend roles (memory tuning of JVMs is a real ops concern).

Region	What lives there	Notes
Heap → Young Gen	Newly allocated objects (Eden, S0, S1)	Most objects die young; GC'd often (minor GC)
Heap → Old Gen	Survivors from Young Gen	GC'd less often (major GC); slower
Metaspace	Class metadata (was PermGen pre-Java 8)	Off-heap; grows as needed
Stack (per thread)	Method frames, locals, refs to heap	Small, fixed-size per thread
PC Register (per thread)	Current bytecode address	Trivially small
Native method stack	JNI / native calls	Separate from JVM stack
Code cache	JIT-compiled native code	Tuned via -XX:ReservedCodeCacheSize

★ **Interview Gold: Why does Young/Old split exist?** Empirical fact: most Java objects die within milliseconds of creation (request-scoped data, temp strings). It's cheaper to GC a small Young Gen frequently than to scan the whole heap. Old Gen is for the survivors — long-lived caches, static config, etc. This is the **generational hypothesis**, and it's the foundation of G1, ZGC, Shenandoah, and every modern garbage collector.

Common JVM interview questions

- **What's -Xmx vs -Xms?** Xmx = max heap. Xms = initial heap. Setting them equal avoids resizing pauses.
- **Stack overflow vs heap overflow?** Stack: deep recursion. Heap: OutOfMemoryError from too many live objects.
- **Why is GC a 'stop-the-world' event?** Some GC phases need to scan refs that may change. To do that safely, application threads are paused. Modern GCs (ZGC, Shenandoah) minimize this to sub-ms pauses.
- **Where do String constants live?** In the heap (since Java 7), in a special interned region.

12.5 Containers vs VMs — The OS View

MUST KNOW HIGH FREQ AMAZON FAV SERVICE-BASED FAV

Both isolate workloads, but at very different OS layers. Get this distinction crisp:

Aspect	Virtual Machine	Container
What's virtualized	Hardware (full guest OS)	OS view (shared host kernel)
Startup	Seconds–minutes	Milliseconds
Per-instance overhead	Full guest kernel + OS	Just process metadata
Density	10s–100s per host	100s–1000s per host
Isolation strength	Strong (hypervisor)	Weaker (shared kernel)
Built from	Hypervisor (KVM, VMware)	Namespaces + cgroups (Linux)
Kernel exploit blast radius	Limited to one VM	Potentially all containers

★ **Interview Gold:** Containers are NOT a new kernel feature. They're a packaging convention on top of two existing Linux primitives — **namespaces** (isolate what a process sees) and **cgroups** (limit what it can use). Docker is essentially: 'standard image format + CLI wrapper + lifecycle daemon' over these primitives. Knowing this is the difference between sounding like a user and sounding like an engineer.

12.6 Namespaces & cgroups (The Container Building Blocks)

RARE BUT IMPRESSIVE SERVICE-BASED FAV

Linux Namespaces — "what you can see"

A namespace wraps a global system resource in an abstraction that makes it appear to the process as if it had its own isolated instance. Linux currently has 8 types of namespace:

Namespace	Isolates	Example effect
PID	Process IDs	Container sees its own PID 1; can't see host processes
NET	Network interfaces, ports, routes	Each container can bind port 80 independently
MNT	Mount points / filesystem view	Each container has its own root filesystem
UTS	Hostname & domain name	Each container can have its own hostname
IPC	SysV IPC, POSIX msg queues	Container can't see another's shared memory
USER	UIDs and GIDs	Root in container = unprivileged on host
CGROUP	cgroup root view	Container sees its own cgroup hierarchy
TIME	Boot and monotonic clocks	Containers can have different uptime

cgroups — "what you can use"

Control groups (**cgroups**) limit and account for resource usage of a group of processes. The kernel tracks CPU, memory, I/O bandwidth, network bandwidth, and PIDs per cgroup. Setting --memory=512m on docker run ultimately writes 512×1024×1024 into a memory cgroup file; if the container's processes

exceed it, the kernel OOM-kills them inside the container.

■ ***Analogy:*** *Namespaces are the walls of an apartment — each tenant only sees their own rooms. Cgroups are the building's metered utilities — each apartment gets a quota of electricity, water, and Wi-Fi bandwidth. Together they make the apartment feel like a self-contained house, even though it shares the building's plumbing (the kernel).*

12.7 Copy-on-Write in Docker Image Layers

RARE BUT IMPRESSIVE SERVICE-BASED FAV

A Docker image is a stack of **layers**, each a read-only filesystem diff. When you start a container, Docker adds a thin read-write layer on top. All layers are presented as one unified filesystem to the container via a **union mount** (overlayfs is the modern default on Linux).

The magic: layers are **shared between containers**. If 100 containers all use the same node:20-alpine base, only one copy of those layers exists on disk. When a container modifies a file, overlayfs **copies it up** to the writable layer first (copy-on-write), leaving the shared layer untouched.

```
# Image layers (read-only, shared):
[ alpine base       ]  <-- shared by 100 containers
[ node binaries     ]  <-- shared
[ npm install layer ]  <-- shared
[ app code COPY     ]  <-- shared
[ container 1 rw    ]  <-- private, COW writes land here
[ container 2 rw    ]  <-- private
...
```

★ **Interview Gold: Why does npm install in your Dockerfile run before COPY . .?** Because Docker caches layers. If you copy your source code first, every code change invalidates the npm install layer's cache and forces re-install. Putting package.json + npm install first means npm install is cached as long as your dependencies don't change. This is OS-level CoW knowledge translating directly to 10× faster builds.

12.8 mmap in Databases

RARE BUT IMPRESSIVE

Many databases use mmap() to map their data files directly into the process's address space, treating gigabytes of data as if it were a giant array in memory. The OS handles paging in and out via virtual memory — the database doesn't manage its own page cache.

Users of this technique:

- **SQLite** can optionally use mmap for read-heavy workloads.
- **LMDB** (Lightning Memory-Mapped Database) is mmap-native. Used by OpenLDAP, Bitcoin Core's wallet.
- **MongoDB** used mmap as its primary storage engine through 3.0 (later replaced by WiredTiger).
- **LevelDB / RocksDB** use mmap for read-only SST files.

★ **Interview Gold: Pros of mmap:** zero-copy reads (kernel-managed page cache becomes your buffer pool), simpler code, OS does the LRU for you, multiple processes share pages naturally. **Cons:** no control over eviction (the OS may evict your hot pages under memory pressure), I/O latency hidden behind opaque page faults (hard to instrument), Postgres famously published *'mmap is bad for us'* and uses a custom buffer pool. Both choices have valid scenarios.

■ ***Analogy:*** *Using mmap is like keeping your library of books on your living room shelves and just walking over to grab one when you need it — the OS is your librarian, fetching from the basement (disk) when needed. A custom buffer pool is like keeping a small personal cabinet on your desk that YOU manage — more work, but you decide exactly what stays in arm's reach.*

12.9 Kernel Bypass Basics

RARE BUT IMPRESSIVE

For ultra-low-latency systems (high-frequency trading, 100 Gbps networking), even a syscall is too slow. **Kernel bypass** techniques let user-space code talk directly to hardware, skipping the kernel's network stack entirely. The headline names:

- **DPDK (Data Plane Development Kit)**: poll-mode NIC drivers in user space. No interrupts, no context switches, no copying. Used by virtual switches and load balancers.
- **RDMA (Remote DMA)**: NIC reads/writes remote machine's memory directly, bypassing both kernels. Used in HPC, storage networks, fast distributed databases (e.g., FaRM).
- **io_uring (Linux 5.1+)**: an asynchronous syscall interface using shared ring buffers between user space and kernel — submit thousands of I/O operations with one syscall. Major modern win.
- **eBPF**: programmable kernel-level hooks. Not bypass exactly, but lets user-defined code run in kernel without leaving it. Powers modern observability (bcc, bpftrace) and networking (Cilium).

★ **Interview Gold:** You almost certainly won't be asked to *implement* any of these as a fresher. But mentioning io_uring or eBPF when discussing 'how would you make this faster?' instantly signals depth. Interviewers who hear it light up — they want people who read past the syllabus.

12.10 NUMA Awareness

RARE BUT IMPRESSIVE

NUMA = Non-Uniform Memory Access. Modern multi-socket servers have multiple memory controllers, one per CPU socket. Memory attached to socket 0 is fast for cores on socket 0, but slower if accessed from socket 1 (because the request travels over the inter-socket interconnect). The OS exposes this via **NUMA nodes**.

Why it matters:

- A thread pinned to socket 0 using memory allocated on socket 1 may run 30–50% slower for memory-heavy code.
- Linux tries to allocate memory on the same NUMA node as the thread that touched it first (*first-touch policy*). If your initialization thread runs on a different node than your worker threads, you get bad locality.
- Tools: numactl --hardware shows your nodes; numactl --membind=0 --cpunodebind=0 ./app pins both.

★ **Interview Gold:** Why this is impressive: 99% of freshers don't even know NUMA exists. Saying *'on a NUMA box we'd want to pin worker threads to their local memory node to avoid cross-socket latency'* in a system design interview is a credibility multiplier. You don't need to design for it — just naming it correctly shows you know what runs your code at scale.

■ ***Analogy:*** *NUMA is your fridge vs your neighbor's fridge. Both have food. Walking to your fridge takes 2 seconds; walking to your neighbor's takes 30. If you stored your lunch in the wrong fridge, every bite*

costs you 30 seconds. NUMA-aware scheduling = the OS makes sure your lunch goes in YOUR fridge by default.

12.11 Section Recap — What to say in interviews

If asked '*tell me something interesting about modern systems*' — pick one of these and have a 2-minute answer ready:

- **'Most modern servers don't thread-per-connection — they use event loops backed by epoll/kqueue, which is how Nginx serves 10K connections from one process.'**
- **'Go's goroutines look synchronous but the runtime uses epoll under the hood — that's the M:N model.'**
- **'Containers aren't VMs — they're just namespaces + cgroups on a shared kernel, which is why they start in milliseconds.'**
- **'Docker layers are read-only and shared via overlayfs copy-on-write — that's why a base image of 1 GB doesn't cost 100 GB across 100 containers.'**
- **'JVM's generational GC exists because most Java objects die young — there's no point scanning the whole heap when 99% of the garbage is in Eden.'**

★ **Interview Gold:** Last tip: when an interviewer asks how to make something faster, the modern-systems answer is almost always *'reduce syscalls, reduce copies, reduce context switches'*. Every technique in this chapter is one of those three in disguise.

13. Interview Cheatsheet & Rapid-Fire

This chapter is your **last-day revision pack**. Skim before the interview. Each section is designed to be re-read fast and answer-ready. Don't memorize — internalize the pattern.

13.1 The 50 most-asked fresher OS questions

Process / Thread (10)

1. **What is an OS?** Software that manages hardware resources and provides services to programs.
2. **Process vs Program?** Program = static file. Process = running instance.
3. **Process vs Thread?** Process has own address space; threads share it. Thread switching is cheaper.
4. **States of a process?** New, Ready, Running, Waiting, Terminated.
5. **What's in a PCB?** PID, state, PC, registers, memory info, FDs, scheduling info.
6. **What does fork() return?** 0 in child, child's PID in parent, –1 on failure.
7. **fork-exec pattern?** Parent forks, child calls exec to run a new program. Used by shells.
8. **Zombie vs orphan?** Zombie = child exited, parent didn't wait. Orphan = parent died first.
9. **User vs kernel thread?** User-level managed by library (kernel sees one process); kernel-level scheduled by OS.
10. **Context switch?** Save current thread's state in PCB, load next thread's state. Pure overhead.

Scheduling (8)

1. **Preemptive vs non-preemptive?** Preemptive can forcibly take CPU; non-preemptive can't.
2. **Which gives min avg WT?** SJF/SRTF (provably optimal).
3. **Which is fairest?** Round Robin.
4. **What is starvation? Fix?** Low-priority process never runs. Fix with aging.
5. **Convoy effect?** One long process delays all short ones in FCFS.
6. **How to choose TQ in RR?** Too small = overhead; too large = behaves like FCFS. ~10–100 ms.
7. **TAT and WT formulas?** TAT = CT – AT, WT = TAT – BT.
8. **MLFQ in one line?** Multiple queues with different priorities; processes move based on behavior.

Synchronization & Deadlock (10)

1. **Race condition?** Outcome depends on thread interleaving. Fix with synchronization.
2. **Mutex vs semaphore?** Mutex has ownership (only locker unlocks). Semaphore is a counter, anyone can signal.
3. **Binary vs counting semaphore?** Binary = 0/1. Counting = manages N resources.
4. **Spinlock vs mutex?** Spinlock busy-waits; better for very short CS. Mutex sleeps; better for longer CS.
5. **4 deadlock conditions?** Mutual exclusion, Hold & Wait, No Preemption, Circular Wait.
6. **Difference: prevention vs avoidance?** Prevention breaks one of the 4 conditions structurally. Avoidance (Banker's) dynamically checks for safe state.

7. Banker's algorithm idea? Grant request only if resulting state is safe (some completion order exists).

8. Deadlock vs starvation vs livelock? Deadlock = blocked forever. Starvation = unfairly skipped. Livelock = busy doing nothing useful.

9. Producer-consumer essentials? Two semaphores (empty, full) + mutex protecting buffer.

10. Reader-writer? Multiple readers OK; writer needs exclusive.

Memory & Virtual Memory (12)

1. Logical vs physical address? Logical = what program sees. Physical = actual RAM location. MMU translates.

2. Internal vs external fragmentation? Internal = wasted space inside an allocation. External = total free space scattered into unusable holes.

3. Paging vs segmentation? Paging = fixed-size pages (no ext frag). Segmentation = variable-size logical units.

4. What's a page table? Per-process map of virtual page → physical frame.

5. What's a TLB? Small hardware cache of recent page-table entries.

6. TLB hit ratio formula? EAT = h(t+m) + (1–h)(t+2m).

7. What's a page fault? Access to a page not currently in memory; OS handles by loading from disk.

8. Why demand paging? Load only needed pages; smaller startup, ability to run programs > RAM.

9. Belady's anomaly? FIFO can fault MORE with more frames. Counterintuitive but real.

10. LRU vs Optimal? Optimal evicts the page used farthest in future (unimplementable; benchmark). LRU evicts least recently used (good practical approximation).

11. Thrashing? Excessive paging — CPU mostly idle waiting for disk. Fix: reduce multiprogramming.

12. Copy-on-write? Parent and child share pages after fork; copy made only when one writes.

File System & I/O (10)

1. What's an inode? Metadata struct for a file: owner, perms, timestamps, data block pointers. No filename.

2. Hard link vs symlink? Hard = extra dir entry to same inode. Symlink = file containing a path.

3. File allocation methods? Contiguous, linked, indexed (Unix uses combined indexed with multi-level).

4. What is journaling? Log changes before applying; replay log after crash to restore consistency.

5. What is mmap? Map a file into address space; access it like memory.

6. Disk access time components? Seek + rotational latency + transfer time. Seek dominates on HDDs.

7. Best general-purpose disk scheduler? C-SCAN or C-LOOK — fair, low total seek.

8. Polling vs interrupts vs DMA? Polling wastes CPU. Interrupts react. DMA does transfer without CPU.

9. What is spooling? Buffer I/O on disk so slow devices can be fed from a queue.

10. Blocking vs non-blocking I/O? Blocking suspends caller. Non-blocking returns immediately with status.

13.2 One-liner definitions (rapid-fire)

Term	One-line definition
System call	Controlled trap into kernel from user code.
Trap	Synchronous exception triggered by an instruction.
Interrupt	Asynchronous signal from a device.
Kernel	Core of OS that runs in privileged mode.
Daemon	Background process not attached to a terminal.
Dispatcher	OS component that performs the context switch.
Scheduler	Decides which process runs next.
Long/Short/Medium-term scheduler	Long: admits to system. Short: picks next on CPU. Medium: swaps out.
Critical section	Code accessing shared resource; needs mutual exclusion.
Atomic operation	Indivisible — either fully done or not at all.
Cache coherence	Keeping multi-core caches consistent.
Page	Fixed-size chunk of virtual memory.
Frame	Fixed-size chunk of physical memory.
Swap space	Disk area used to hold paged-out memory.
Working set	Pages a process has used in the last Δ accesses.
Locality	Tendency to access nearby/recently-used memory.
TRIM	OS tells SSD which blocks are no longer in use.
NUMA	Non-Uniform Memory Access — memory closer to some CPUs is faster.
RAID	Combining disks for performance/redundancy (RAID 0/1/5/6/10).

13.3 Numerical formula sheet

Scheduling

- TAT = CT – AT
- WT = TAT – BT
- RT = First_CPU_Time – AT
- Throughput = jobs_completed / time

Memory / TLB

- EAT (TLB) = h(t + m) + (1 – h)(t + 2m)

- EAT (page fault) = (1 – p)·m + p·page_fault_service_time
- Page table size = (virt_addr_space / page_size) × PTE_size
- Internal fragmentation ≈ (page_size / 2) per allocation (avg)

Disk

- Disk access time = seek_time + rotational_latency + transfer_time
- Average rotational latency = 1 / (2 × rpm) × 60 seconds → at 7200 RPM ≈ 4.17 ms
- Total head movement = Σ |next_cyl – current_cyl|

13.4 Diagrams to draw on the whiteboard

If you have a whiteboard or paper in the interview, these diagrams will earn you points. Practice drawing them in under 30 seconds each.

- **Process state diagram** — 5 states with transition arrows.
- **Memory layout of a process** — text/data/BSS/heap↑ ... ↓stack.
- **Address translation in paging** — virtual address split into page# + offset, page table lookup, physical frame# + offset.
- **Gantt chart** for any scheduling problem.
- **Resource Allocation Graph** — circles (processes), squares (resources), request/assignment arrows.
- **Inode tree** — direct, single, double, triple indirect pointers.
- **Disk geometry** — platter, track, sector, cylinder.

13.5 Common follow-up traps

Interviewers rarely ask just the question — they ask the follow-up after you answer. Be ready:

- Q: *'Difference between process and thread?'* Follow-up: *'Then why use processes at all?'* → Isolation (a crash in one doesn't kill the others); security boundaries.
- Q: *'What is a deadlock?'* Follow-up: *'Give an example from code you'd write.'* → Two locks acquired in opposite order by two threads. Draw the scenario.
- Q: *'What is virtual memory?'* Follow-up: *'What's the cost?'* → Page table memory overhead, TLB miss latency, page fault disk I/O.
- Q: *'Mutex vs semaphore?'* Follow-up: *'When would you pick a semaphore over a mutex?'* → When managing a pool of N identical resources, or signaling between threads (e.g., producer-consumer).
- Q: *'Explain paging.'* Follow-up: *'What's the size of the page table for a 32-bit system with 4 KB pages?'* → 2^{20} entries × 4 bytes = 4 MB per process. Then they ask how multi-level paging fixes it.

13.6 Behavioral angle — 'how have you used OS concepts?'

Even fresher SDE interviews sometimes probe whether you connect OS to your code. Practice having one or two of these ready:

- **Threading in a project**: 'I used a thread pool to handle concurrent HTTP requests in my internship project — sized to roughly the number of cores to avoid context switching overhead.'

• **Memory profiling**: 'I noticed my app's RSS growing — used valgrind / heaptrack and found a leak in unfreed buffers in a hot path.'

• **Race condition debugging**: 'Two threads were updating a shared counter without a lock; intermittent off-by-N bugs in tests. Fixed with a std::atomic / AtomicLong.'

• **Choosing a data structure for cache friendliness**: 'Switched from a linked list to a vector because cache misses on pointer chasing were killing throughput.'

13.7 The 5-minute pre-interview revision

If you have literally 5 minutes before the interview, re-read just this:

1. **Process vs thread** — own address space vs shared; cost of context switch.
2. **4 deadlock conditions** — Mutual exclusion, Hold & Wait, No Preemption, Circular Wait.
3. **Scheduling formulas** — TAT = CT – AT, WT = TAT – BT.
4. **Paging mechanics** — virtual page → page table → physical frame; TLB caches translations.
5. **Page fault** — access to non-resident page; trap to OS; load from disk; resume.
6. **Producer-consumer pattern** — empty semaphore, full semaphore, mutex.
7. **Mutex has ownership; semaphore is a counter.**
8. **Belady's anomaly** — FIFO can fault more with more frames.
9. **Thrashing** — too much paging, no useful work; reduce multiprogramming.
10. **fork() returns 0 in child, PID in parent.**

★ **Interview Gold:** Final advice: when you don't know an answer, **think out loud**. Interviewers value reasoning more than recall. Start from first principles — 'the OS is mediating shared resources here, so...' — and you'll often arrive at the answer or a defensible approximation. Good luck.

— End of OS Placement Prep —